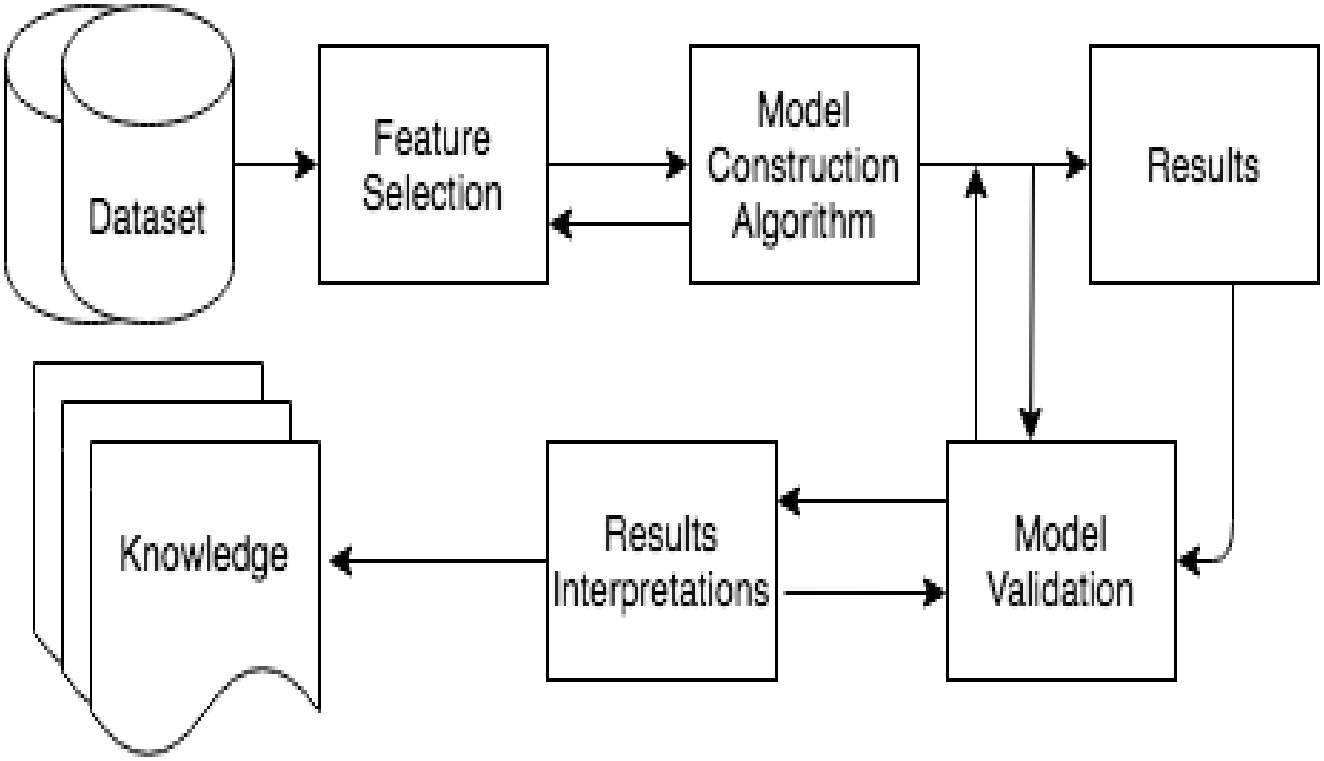
Classification Model

- Classification: Basic Concepts
- Rule-Based Classification
- Decision Tree Induction
- Bayes Classification Methods 
- kNN Classifier
- Performance Evaluation
- Ensemble Classification
- Summary

Basic Steps of Data Analysis



Bayesian Classifier

Bayes' Theorem

- Theorem: If $A_1, A_2, \ldots, A_n$ be a given set of n pairwise mutually exclusive events, one of which certainly occurs, i.e., $A_i A_j = \emptyset \ (i \neq j; i, j = 1,2, \ldots, n)$ and $A_1 + A_2 + \cdots + A_n = S$, then for any arbitrary event X,

(i) $P(X) = P(A_1)P(X/A_1) + P(A_2)P(X/A_2) + \cdots + P(A_n)P(X/A_n)$

(ii) Bayes' theorem: if P(X) ≠ 0,

$$P(A_i/X) = \frac{P(A_i)P(X/A_i)}{P(A_1)P(X/A_1) + P(A_2)P(X/A_2) + \cdots + P(A_n)P(X/A_n)} \ (for\ i = 1, 2, \ldots, n)$$

Proof: For any event X, we have X=SX=$(A_1 + A_2 + \cdots + A_n)X = A_1X + A_2X + \cdots + A_nX$

Since, $(A_iX)(A_jX) = A_iA_jX = \emptyset X = \emptyset, for\ i \neq j; i, j = 1,2, \ldots, n$

Therefore, $A_1X, A_2X, \ldots, A_nX$ are pairwise mutually exclusive events, and hence

$P(X) = P(A_1X + A_2X + \cdots + A_nX) = P(A_1X) + P(A_2X) + \cdots + P(A_nX)$ [Addition Rule]

Since, $P(A_iX) = P(A_i)P(X/A_i)$ [Multiplication Rule]

Therefore, $P(X) = P(A_1)P(X/A_1) + P(A_2)P(X/A_2) + \cdots + P(A_n)P(X/A_n)$ [(i) is proved]

Bayes' Theorem

We have already proved that $P(X) = P(A_1)P(X/A_1) + P(A_2)P(X/A_2) + \cdots + P(A_n)P(X/A_n)$

Now we have to prove the Bayes' theorem: i.e., if P(X) ≠ 0,

$$P(A_i/X) = \frac{P(A_i)P(X/A_i)}{P(A_1)P(X/A_1) + P(A_2)P(X/A_2) + \cdots + P(A_n)P(X/A_n)} \; (for\ i = 1, 2, \ldots, n)$$

Proof: $P(A_iX) = P(A_i)P(X/A_i)$ [Multiplication Rule]

Also, $P(A_iX) = P(X)P(A_i/X)$ [Multiplication Rule]

Hence, if P(X) ≠ 0,

$$P(A_i/X) = \frac{P(A_iX)}{P(X)} = \frac{P(A_i)P(X/A_i)}{P(X)} = \frac{P(A_i)P(X/A_i)}{P(A_1)P(X/A_1) + P(A_2)P(X/A_2) + \cdots + P(A_n)P(X/A_n)}$$

Thus the Bayes' theorem is proved.

Example on Bayes' Theorem

Example-1: There are three identical urns containing white and black balls. The first urn contains 2 white and 3 black balls, the second urn 3 white and 5 black balls, and the third urn 5 white and 2 black balls. An urn is chosen at random, and a ball is drawn from it. If the ball drawn is white, what is the probability that the second urn is chosen?

Solution: ?

Example on Bayes' Theorem

Example-1: There are three identical urns containing white and black balls. The first urn contains 2 white and 3 black balls, the second urn 3 white and 5 black balls, and the third urn 5 white and 2 black balls. An urn is chosen at random, and a ball is drawn from it. If the ball drawn is white, what is the probability that the second urn is chosen?

Solution:

- Let A_1= the event that the ball is drawn from the first urn, A_2= the event that the ball is drawn from the second urn, and A_3= the event that the ball is drawn from the third urn.
- The events A_1, A_2, and A_3 are pairwise mutually exclusive, and one of these necessarily occurs.
- $P(A_1)$ is the probability that 1^{st} urn is chosen, and so on. So, $P(A_1)=P(A_2)=P(A_3)=1/3$
- Let X=the event that ball drawn is white. So, $P(X/A_1)=2/5$, $P(X/A_2)=3/8$, $P(X/A_3)=5/7$.

- We have to compute $P(A_2/X)$

Example-1 Cont…

Compute $P(A_2/X)$

- $P(X)=P(A_1)P(X/A_1) + P(A_2)P(X/A_2)+P(A_3)P(X/A_3)$

  $=(1/3)(2/5)+ (1/3)(3/8)+ (1/3)(5/7)$

  $=(1/3)[2/5+3/8+5/7]$

  $=(1/3)(417/280) = 139/280$

- So, By Bayes' theorem:

$P(A_2/X) = [P(X/A_2)\ P(A_2)]/\ P(X)$

$= [(3/8)(1/3)] / (139/280)$

$=35/139$ (Ans.)

Example-2: LAPLACE'S URN PROBLEM

- There are (N+1) identical urns marked 0,1,2,…,N, each of which contains N white and black balls. The i-th urn contains i black and N-i white balls (i=0,1,2,…,N). An urn is chosen at random, and n random drawings are made from it, the ball drawn being always replaced. If all the n balls turn out to be black, what is the probability that the next ball drawn will also be black?

Solution:

- Let A_i =the event that the i-th urn is chosen. So, the events $A_0, A_1, \ldots, A_N$ are pairwise mutually exclusive, one of which certainly occurs. Obviously, $P(A_i) = \frac{1}{N+1}$, for i=0,1,2,…,N.

Example-2 Cont..: LAPLACE'S URN PROBLEM

- Let X = the event that all n balls drawn are black. So,

$$P(X/A_i) = \frac{i^n}{N^n}$$

- $$P(X) = P(A_0)P(X/A_0) + P(A_1)P(X/A_1) + \cdots + P(A_N)P(X/A_N) = \frac{1}{N+1}\sum_{i=0}^{N}(\frac{i}{N})^n \quad \text{.....(1)}$$

- Let Y=the event that (n+1)th ball drawn is black. The XY=the event that all the n+1 balls drawn are black. So, replacing n by n+1 in (1), we get : $P(XY) = \frac{1}{N+1}\sum_{i=0}^{N}(\frac{i}{N})^{n+1}$

- Hence the required probability is:

$$P(Y/X) = \frac{P(XY)}{P(X)} = \frac{\sum_{i=0}^{N}(\frac{i}{N})^{n+1}}{\sum_{i=0}^{N}(\frac{i}{N})^n} \qquad \text{.....(2)}$$

Example-2 Cont..: LAPLACE'S URN PROBLEM

- If N is very large: In this case,

$$P(X) \cong \frac{1}{N}\sum_{i=0}^{N}(\frac{i}{N})^n \cong \int_0^1 x^n dx = \frac{1}{n+1}$$

$$P(XY) \cong \frac{1}{N}\sum_{i=0}^{N}(\frac{i}{N})^{n+1} \cong \int_0^1 x^{n+1} dx = \frac{1}{n+2}$$

- Therefore,

$$P(Y/X) = \frac{P(XY)}{P(X)} = \frac{\sum_{i=0}^{N}(\frac{i}{N})^{n+1}}{\sum_{i=0}^{N}(\frac{i}{N})^{n}} = \frac{n+1}{n+2}$$

- This is called the law of succession of Laplace.

$$\int_a^b f(x)dx = \lim_{\substack{n\to\infty \\ h\to 0}} h\left[f(a)+f(a+h)+f(a+2h)+.....+f(a+(n-1)h)\right]$$

, where $h = \frac{b-a}{n}$

(or)

$$\int_a^b f(x)dx = \lim_{\substack{n\to\infty \\ h\to 0}} \sum_{r=1}^{n} h\, f(a+rh), \text{ where } h = \frac{b-a}{n}$$

Example 2.81

Evaluate the integral as the limit of a sum: $\int_0^1 x\,dx$

Solution:

$$\int_a^b f(x)dx = \lim_{\substack{n\to\infty \\ h\to 0}} \sum_{r=1}^{n} h\, f(a+rh)$$

Here $a=0,\ b=1,\ h=\frac{b-a}{n}=\frac{1-0}{n}=\frac{1}{n}$ and $f(x)=x$

Now $f(a+rh)=f\left(0+\frac{r}{n}\right)=f\left(\frac{r}{n}\right)=\frac{r}{n}$

On substituting in (1) we have

$$\int_0^1 x\,dx = \lim_{n\to\infty}\sum_{r=1}^{n}\frac{1}{n}.\frac{r}{n}$$

$$= \lim_{n\to\infty}\frac{1}{n^2}\sum_{r=1}^{n} r$$

$$= \lim_{n\to\infty}\frac{1}{n^2}\,\frac{n(n+1)}{2}$$

$$= \lim_{n\to\infty}\frac{1}{n^2}\cdot\frac{n^2\left(1+\frac{1}{n}\right)}{2}$$

$$= \frac{1+0}{2}=\frac{1}{2}$$

$$\therefore \quad \int_0^1 x\,dx = \frac{1}{2}$$

Naïve Bayesian Classification

- A statistical classifier: performs *probabilistic prediction, i.e.,* predicts class membership probabilities
- Foundation: Based on Bayes' Theorem.
- Performance: A simple Bayesian classifier, *naïve Bayesian classifier*, has comparable performance with decision tree and selected neural network classifiers

Bayesian Classification

$$P(B|A) = \frac{P(A|B).P(B)}{P(A)}$$ Bayes' Theorem

P(B|A) = Posterior Probability

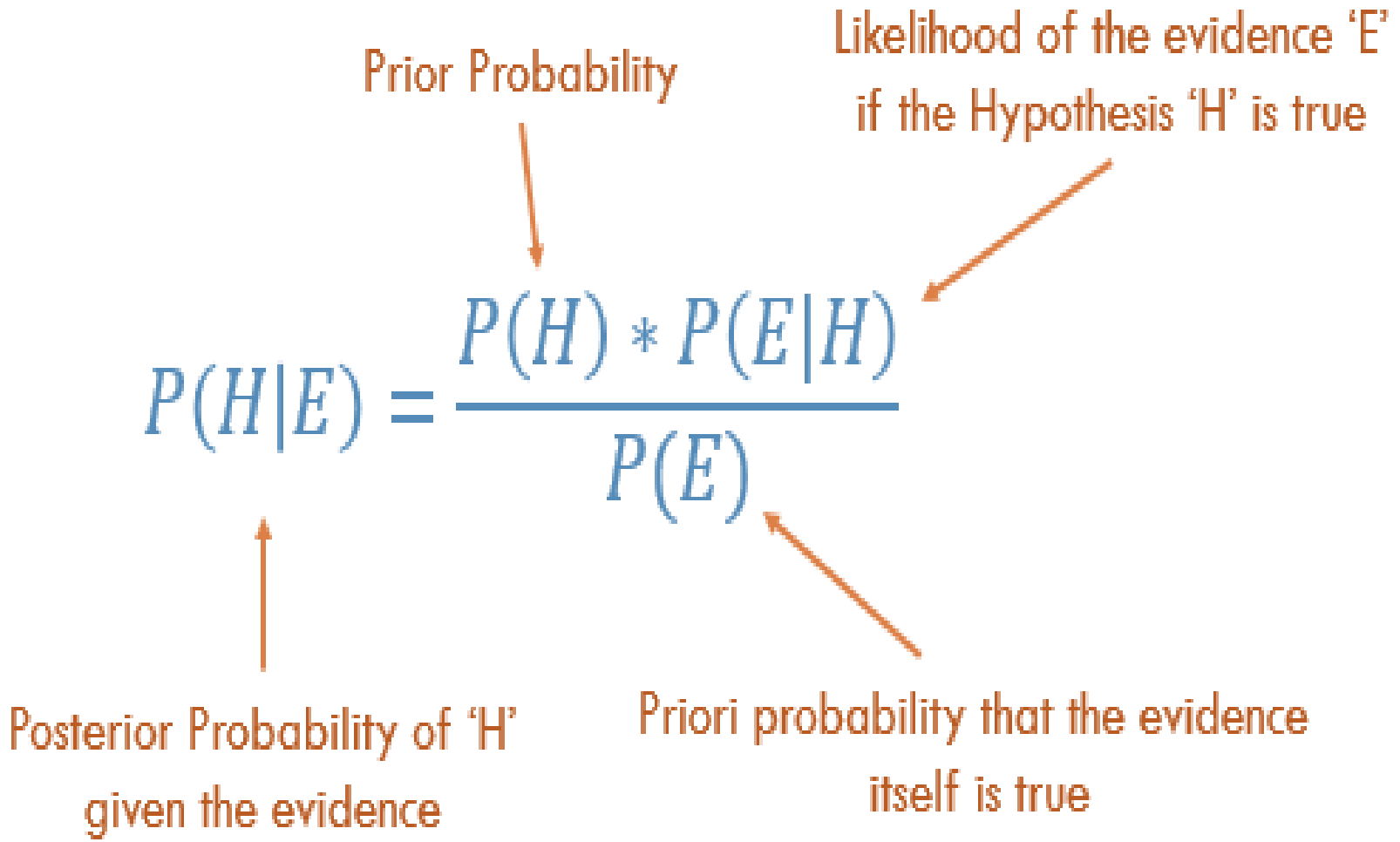
P(A|B) = Likelihood

P(B) = Prior Probability

-> Posterior refers to probability obtained *after* evidence (data) has been taken into consideration.

-> Prior refers to probability *before* evidence (data) has been taken into consideration.

Bayesian Classification



Bayes' Theorem: Basics

- Bayes' Theorem: $$P(H \mid \mathbf{X}) = \frac{P(\mathbf{X} \mid H)P(H)}{P(\mathbf{X})} = P(\mathbf{X} \mid H) \times P(H) / P(\mathbf{X})$$
  - Let **X** be a data sample: class label is unknown
  - Let H be a *hypothesis* that X belongs to class C
  - Classification is to determine P(H|**X**), (i.e., *posteriori probability):* the probability that the hypothesis holds given the observed data sample **X**
  - P(H) (*prior probability*): the initial probability
    - E.g., **X** will buy computer, regardless of age, income, student, credit_rating.
  - P(**X**): probability that sample data is observed
  - P(**X**|H) : the probability of observing the sample **X**, given that the hypothesis holds
    - E.g., Given that **X** will buy computer, the prob. that X is 31..40, medium income

Prediction Based on Bayes' Theorem

- Given training data **X**, *posteriori probability of a hypothesis* H, P(H|**X**), follows the Bayes' theorem

$$P(H|\mathbf{X}) = \frac{P(\mathbf{X}|H)P(H)}{P(\mathbf{X})} = P(\mathbf{X}|H) \times P(H) / P(\mathbf{X})$$

- Predicts **X** belongs to C_i iff the probability $P(C_i|\mathbf{X})$ is the highest among all the $P(C_k|X)$ for all the *k* classes
- Practical difficulty: It requires initial knowledge of many probabilities, involving significant computational cost

Classification Is to Derive the Maximum Posteriori

- Let D be a training set of tuples and their associated class labels, and each tuple is represented by an n-D attribute vector **X** = (x_1, x_2, …, x_n)
- Suppose there are *m* classes C_1, C_2, …, C_m.
- Classification is to derive the maximum posteriori, i.e., the maximal $P(C_i|\mathbf{X})$
- This can be derived from Bayes' theorem

$$P(C_i|\mathbf{X}) = \frac{P(\mathbf{X}|C_i)P(C_i)}{P(\mathbf{X})}$$

- Since P(X) is constant for all classes, so only

$$P(C_i|\mathbf{X}) = P(\mathbf{X}|C_i)P(C_i)$$

needs to be maximized

Naïve Bayes Classifier

- A simplified assumption: attributes are conditionally independent (i.e., no dependence relation between attributes):

$$P(\mathbf{X}|C_i) = \prod_{k=1}^{n} P(x_k|C_i) = P(x_1|C_i) \times P(x_2|C_i) \times ... \times P(x_n|C_i)$$

- This greatly reduces the computation cost: Only counts the class distribution
- If A_k is categorical, $P(x_k|C_i)$ is the # of tuples in C_i having value x_k for A_k divided by $|C_{i,D}|$ (# of tuples of C_i in D)
- If A_k is continous-valued, $P(x_k|C_i)$ is usually computed based on Gaussian distribution with a mean μ and standard deviation σ

$$g(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

and $P(x_k|C_i) = g(x_k, \mu_{C_i}, \sigma_{C_i})$

Naïve Bayes Classifier: Training Dataset

Class:
C1:buys_computer = ‘yes’
C2:buys_computer = ‘no’

Data to be classified:
X = (age <=30,
Income = medium,
Student = yes
Credit_rating = Fair)

age	income	student	credit_rating	buys_comp
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

Naïve Bayes Classifier: An Example

- $P(C_i)$: P(buys_computer = "yes") = 9/14 = 0.643
- To Compute P(X| buys_computer = "yes")
  - P(age = "<=30" | buys_computer = "yes") = 2/9 = 0.222
  - P(income = "medium" | buys_computer = "yes") = 4/9 = 0.444
  - P(student = "yes" | buys_computer = "yes) = 6/9 = 0.667
  - P(credit_rating = "fair" | buys_computer = "yes") = 6/9 = 0.667
- **X = (age <= 30 , income = medium, student = yes, credit_rating = fair)**

$P(X|C_i)$: P(X|buys_computer = "yes") = 0.222 x 0.444 x 0.667 x 0.667 = 0.044

$P(X|C_i)*P(C_i)$: P(X|buys_computer = "yes") * P(buys_computer = "yes") = 0.028

age	income	student	credit_rating	buys_comp
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

Naïve Bayes Classifier: An Example

- $P(C_i)$: P(buys_computer = "no") = 5/14= 0.357
- To Compute P(X|buys_computer = "no")
  - P(age = "<= 30" | buys_computer = "no") = 3/5 = 0.6
  - P(income = "medium" | buys_computer = "no") = 2/5 = 0.4
  - P(student = "yes" | buys_computer = "no") = 1/5 = 0.2
  - P(credit_rating = "fair" | buys_computer = "no") = 2/5 = 0.4
- **X = (age <= 30 , income = medium, student = yes, credit_rating = fair)**

$P(X|C_i)$: P(X|buys_computer = "no") = 0.6 x 0.4 x 0.2 x 0.4 = 0.019

$P(X|C_i)*P(C_i)$: P(X|buys_computer = "no") * P(buys_computer = "no") = 0.007

age	income	student	credit_rating	buys_comp
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

$P(X|C_i)*P(C_i)$: P(X|buys_computer = "yes") * P(buys_computer = "yes") = 0.028

$P(X|C_i)*P(C_i)$: P(X|buys_computer = "no") * P(buys_computer = "no") = 0.007

Therefore, X belongs to class ("buys_computer = yes")

Naïve Bayes Classifier: Comments

- Advantages
  - Easy to implement
  - Good results obtained in most of the cases
- Disadvantages
  - Assumption: class conditional independence, therefore loss of accuracy
  - Practically, dependencies exist among variables
    - E.g., hospitals: patients=> Profile: age, family history, etc.

      Symptoms: fever, cough etc., Disease: lung cancer, diabetes, etc.
    - Dependencies among these cannot be modeled by Naïve Bayes Classifier
- How to deal with these dependencies? Bayesian Belief Networks

Overfitting in Naïve Bayes Classification

- **Naïve Bayes** can suffer from **overfitting**, although it is generally less prone to it compared to more complex models. Overfitting in Naïve Bayes can happen in the following cases:

1. When the Training Data is Noisy or Small

- If the dataset is small, the estimated probabilities may not generalize well to new data.
- The model might assign very high or very low probabilities to certain features, making predictions too **rigid**.

2. When Features are Strongly Correlated

- Naïve Bayes assumes **independence** between features, which is often unrealistic.
- If features are highly correlated, the model might **double-count information**, leading to biased probability estimations and poor generalization.

Overfitting in Naïve Bayes Classification

3. When Using Maximum Likelihood Estimation (MLE)

- If a class-feature combination does not appear in the training set, Naïve Bayes assigns it a **zero probability**.
- This makes the classifier overfit by being overly confident about observed data and failing on unseen examples.

Resolve Overfitting in Naïve Bayes

1. Avoiding the Zero-Probability Problem

- Naïve Bayesian prediction requires each conditional prob. be **non-zero**. Otherwise, the predicted prob. will be zero.

$$P(X \mid C_i) = \prod_{k=1}^{n} P(x_k \mid C_i)$$

- **Use Smoothing (Laplace / Additive Smoothing)**
- Prevents **zero probabilities** by adding a small value (e.g., 1) to all counts.
- Formula for **Laplace Smoothing** $P(w|C) = \frac{count(w, C) + \alpha}{count(C) + \alpha \times |V|}$

- where: α is a smoothing parameter (typically **1**), and |V| is the vocabulary size.

Resolve Overfitting in Naïve Bayes

1. Avoiding the Zero-Probability Problem

- Ex. Suppose a dataset with 1000 tuples, income=low (990), income= medium (0), and income = high (10)
- Use **Laplacian correction** (or Laplacian estimator)
  - *Adding 1 to each case*

    Prob(income = low) = 991/1003

    Prob(income = medium) = 1/1003

    Prob(income = high) = 11/1003
  - The "corrected" prob. estimates are close to their "uncorrected" counterparts

Resolve Overfitting in Naïve Bayes

2. Feature Selection / Dimensionality Reduction

- Remove redundant or highly correlated features to better align with the Naïve Bayes independence assumption.
- Use PCA, LDA, or Mutual Information to select only relevant features.

3. **Use a More Advanced Naïve Bayes Variant**

- Gaussian Naïve Bayes: If features are continuous, assume they follow a Gaussian distribution.
- Multinomial Naïve Bayes: Suitable for text classification.
- Bernoulli Naïve Bayes: Best for binary feature representation.

Resolve Overfitting in Naïve Bayes

4. Bayesian Priors

- Instead of relying purely on observed data, incorporate **prior knowledge** to balance probabilities.
- Helps when dealing with **imbalanced datasets**.

5. Cross-Validation

- Use **k-fold cross-validation** to ensure the model generalizes well across different training sets.
- Helps in choosing the best smoothing parameter (α). (How?)

Cross-validation for tuning α

- K-fold cross-validation helps in selecting the best smoothing parameter (α) for **Laplacian smoothing** in **Naïve Bayes classification**
- It systematically evaluates different values of α on multiple training and validation sets, ensuring optimal performance.
- In Laplacian Smoothing, $\alpha > 0$ prevents zero probability. The effectiveness of smoothing depends on the choice of α. Too small α can lead to underfitting, while too large α can cause over-smoothing.

Cross-validation for tuning α

- **K-fold cross-validation** helps determine the best α as follows:

1. Divide Data into K-Folds

- The dataset is split into K subsets (folds).
- The model is trained on K−1 folds and validated on the remaining fold.
- This process is repeated K times, with each fold serving as validation once.

Cross-validation for tuning α

2. Test Multiple Values of α

- Train the Naïve Bayes model using different values of α (e.g., 0.1, 0.5, 1, 10).
- Compute performance metrics (accuracy, log-likelihood, F1-score) on the validation fold.

3. Average Performance Across Folds

- The performance of each α is averaged over all K iterations.
- This ensures that results are not biased by any specific data split.

4. Select the Best α

- The α yielding the highest average validation performance is chosen as the optimal smoothing parameter.
- This optimally balances generalization and model accuracy.

Cross-validation for tuning α

- K-fold cross-validation helps in **fine-tuning the Laplacian smoothing parameter** α by systematically evaluating different values and selecting the one that minimizes classification error.
- This improves the generalization ability of the Naïve Bayes classifier, ensuring it handles unseen data effectively.

kNN Algorithm

kNN Classifier

- k-nearest neighbours (kNN) algorithm as a type of supervised algorithm which can be used for both classification as well as regression predictive problems.
- There are three categories of learning algorithms:

  i) **Lazy learning algorithm:** kNN is a lazy learning algorithm because it does not have a specialized training phase or model and uses all the data for training while classification

  ii) **Non-parametric learning algorithm:** kNN is also a non-parametric learning algorithm because it does not assume about the distribution of the underlying data (as opposed to other algorithms such as Gaussian Mixture Model (GMM), which assume a Gaussian distribution about the data

  iii) **Eager learning algorithm:** Eager learners, when given a set of training tuples, will construct a generalization model before receiving new (e.g., test) tuples to classify.

kNN Classifier

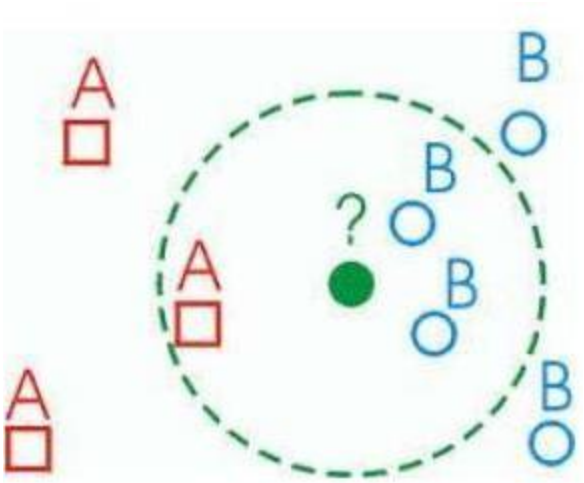
- The kNN algorithm begins with a training dataset made up of examples that are classified into several categories.
- Assume that we have a test dataset containing unlabeled examples that otherwise have the same features as the training data.
- For each example (i.e., record) in the test dataset, kNN identifies k examples in the training data that are the "nearest" in similarity, where k is an integer specified in advance.
- The unlabeled test instance is assigned the class of the majority of the k nearest neighbors.

KNN: Classification Approach

- Locating the unlabeled instance's nearest neighbors requires a distance function, or a formula that measures the similarity between two instances.
- There are many different ways to calculate distance. Traditionally, the kNN algorithm uses Euclidean distance.
- The K-NN algorithm works by finding the K nearest neighbors to a given data point based on a distance metric, such as Euclidean distance.
- The class or value of the data point is then determined by the majority vote (for classification) or average (for regression) of the K neighbors.

KNN: Classification Approach

- Classified by "**MAJORITY VOTES**" for its neighbor classes
  - Assigned to the most common class amongst its *K*-nearest neighbors



KNN: Pseudocode

- Step 1: Determine parameter K = number of nearest neighbors
- Step 2: Calculate the distance between the query-instance and all the training examples.
- Step 3: Sort the distance and determine nearest neighbors based on the k-th minimum distance.
- Step 4:Gather the category Y of the nearest neighbors.
- Step 5: Use simple majority of the category of nearest neighbors as the prediction value of the query instance.

Advantages of kNN

- kNN algorithm is a versatile and widely used machine learning algorithm that is primarily used for its simplicity and ease of implementation.
- It does not require any assumptions about the underlying data distribution.
- It can also handle both numerical and categorical data, making it a flexible choice for various types of datasets in classification and regression tasks.
- It is a non-parametric method that makes predictions based on the similarity of data points in a given dataset.

Advantages Contd..

- K-NN is less sensitive to outliers compared to other algorithms.

- **Few Hyperparameters** – The only parameters which are required in the training of a KNN algorithm are the value of k and the choice of the distance metric which we would like to choose from our evaluation metric.

Disadvantages of the KNN Algorithm

- **Does not scale** – It is a lazy Algorithm. The main significance of this term is that this takes lots of computing power as well as data storage. This makes this algorithm both time-consuming and resource exhausting.
- **Curse of Dimensionality** – The KNN algorithm is affected by the curse of dimensionality which implies the algorithm faces a hard time classifying the data points properly when the dimensionality is too high.
- **Prone to Overfitting** – As the algorithm is affected due to the curse of dimensionality it is prone to the problem of overfitting as well. Hence, generally feature selection as well as dimensionality reduction techniques are applied to deal with this problem.

Why KNN algorithm does not perform well on high-dimensional datasets?

1. Curse of Dimensionality

- As the number of dimensions (**features**) increases, data points become more **sparse** in the feature space.
- The **distance between points** becomes **less meaningful**, making it difficult for KNN to find the nearest neighbors effectively.
- This leads to poor generalization and degraded performance.

2. Increased Computational Complexity

- KNN is a **lazy learner**, meaning it **stores** all training samples and computes distances at query time.
- In high dimensions, computing distances (e.g., **Euclidean distance**) becomes computationally expensive.
- The time complexity for searching nearest neighbors increases significantly.

Why KNN algorithm does not perform well on high-dimensional datasets?

3. Diminishing Differences in Distances

- In high dimensions, the **difference between the closest and farthest points** reduces significantly.
- Most points end up being **almost equidistant**, making it harder to distinguish between nearest and non-nearest neighbors.

4. Increased Noise Sensitivity

- High-dimensional data often contains **irrelevant or redundant features**.
- KNN treats all features equally, meaning noisy or less important features can mislead the distance calculation.
- Feature selection or **dimensionality reduction (PCA, t-SNE, LDA, etc.)** is often needed before applying KNN.

Why KNN algorithm does not perform well on high-dimensional datasets?

5. Memory Overhead

- Since KNN stores all training data, a high-dimensional dataset requires **more storage** and slows down retrieval.

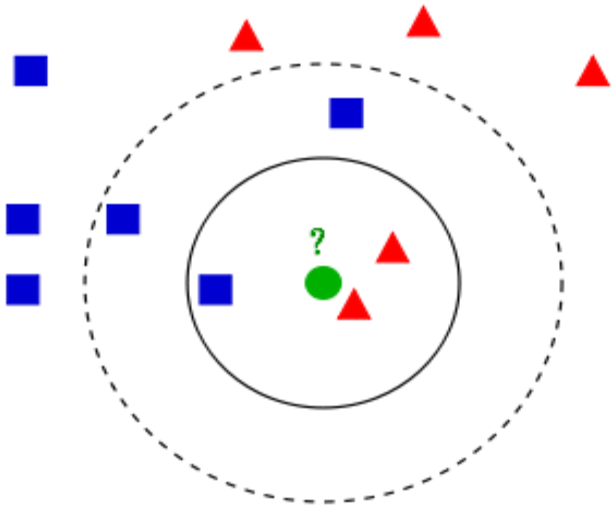
- **Possible Solutions**

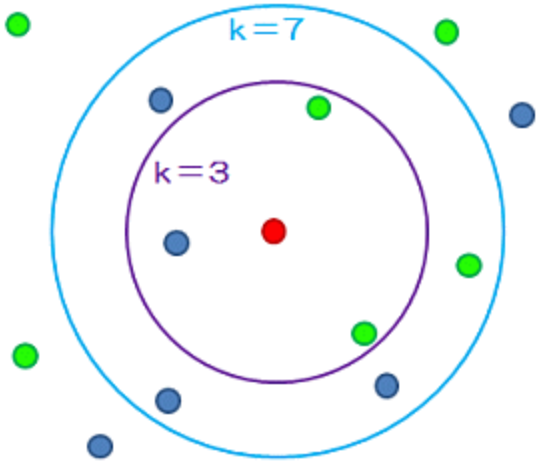
  1. Dimensionality Reduction: Use PCA, LDA, or feature selection techniques to reduce dimensions before applying KNN.

  2. Use Distance Metrics Better Suited for High-Dimensional Data: Cosine similarity or Mahalanobis distance instead of Euclidean distance.

  3. Use Approximate Nearest Neighbors (ANN): Faster alternatives like **KD-Trees, Ball Trees, or Locality-Sensitive Hashing (LSH)** can be used instead of brute-force search.

Variation In kNN





How to Choose the value of k?

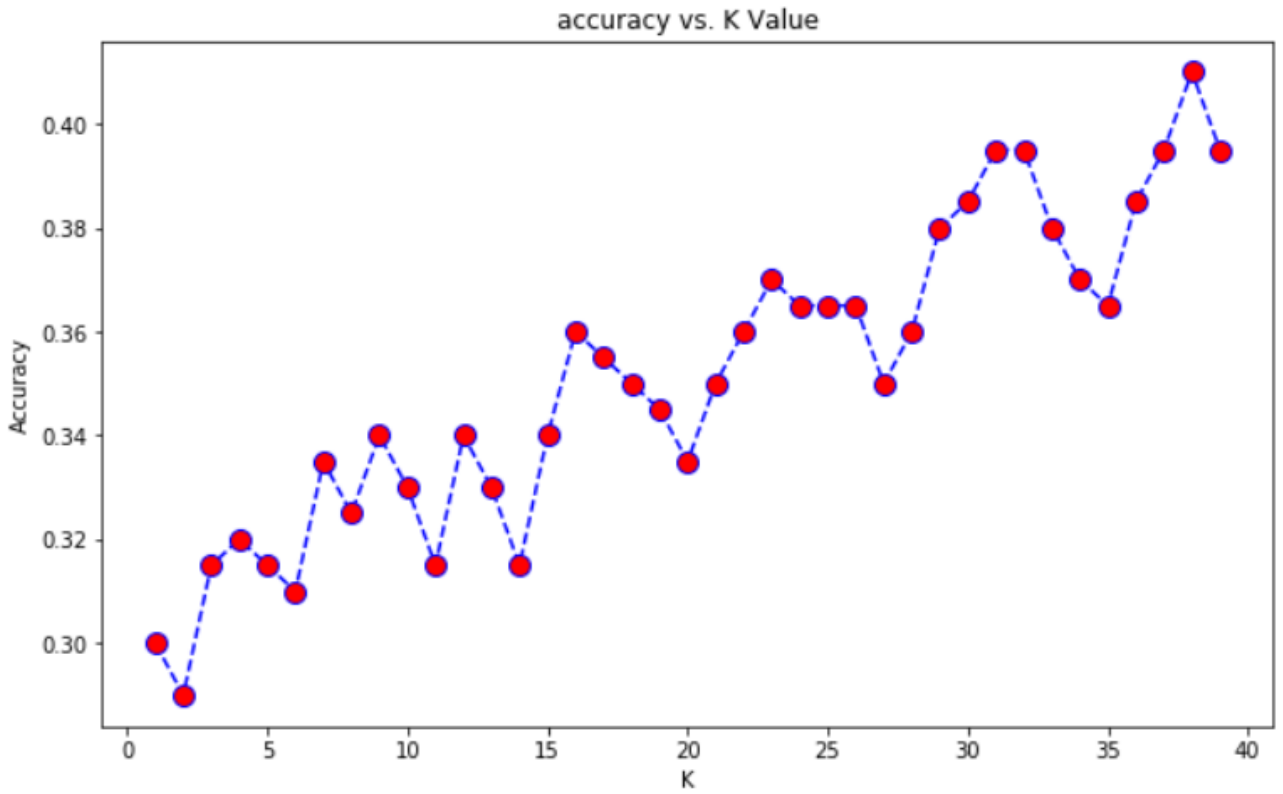
- The value of k is very crucial in the KNN algorithm to define the number of neighbors in the algorithm.
- The value of k in the k-nearest neighbors (k-NN) algorithm should be chosen based on the input data. If the input data has more outliers or noise, a higher value of k would be better.
- It is recommended to choose an odd value for k to avoid ties in classification.

Value of k

- The **small k value isn't suitable** for classification.
- As a rule of thumb, setting k to the square root of the number of training samples can lead to better result. If k becomes 10 after the square root, we can choose either k=9 or k=11 just to make sure that k is odd.
- Use an error plot or accuracy plot to find the most favorable k value.
- kNN performs well with multi-label classes, but you must be aware of the outliers.

How to Choose the value of k?

```
Maximum accuracy:- 0.41 at K = 37
```



- We got the accuracy of **0.41 at K=37**. As we got the minimum error at k=37, so we will get **better efficiency** at that K value.

Is Naïve Bayes a lazy learner?

- The Naive Bayes algorithm is not a *lazy* learner. It is an eager learner. It is different from the nearest neighbor algorithm.
- A real learning takes place for Naive Bayes. The parameters that are learned in Naive Bayes are the *prior probabilities* of different classes, as well as the *likelihood* of different features for each class.
- In the test phase, these learned parameters are used to estimate the probability of each class for the given sample.
- In other words, in Naive Bayes, for each sample in the test set, the parameters determined during training are used to estimate the probability of that sample belonging to different classes.

Is Naïve Bayes a lazy learner?

- For example, $P(c|x) \propto P(c)\ P(x_1|c)\ P(x_2|c)\ \ldots\ p(x_n|c)$, where c is a class and x is a test sample.
- All quantities $P(c)$ and $P(x_i|c)$ are parameters which are determined during training and are used during testing.
- This is similar to NN, but the kind of learning and the kind of applying the learned model is different.

THANK YOU